

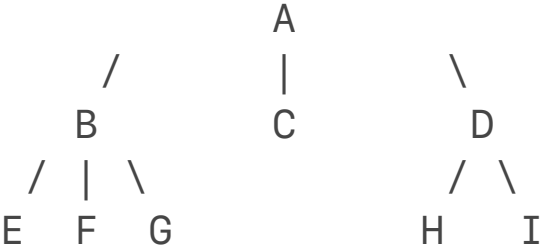
STRUKDAT BAB 8

Tree N-er (N-ary Tree)

1. Pengertian

Tree N-er (N-ary Tree) adalah tree yang setiap node dapat memiliki:
0 sampai N anak
Tidak seperti Binary Tree yang maksimal hanya memiliki:
2 anak

Contoh:



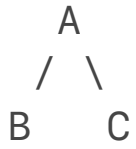
Node:
A
memiliki:
3 anak

2. Perbedaan Binary Tree dan N-ary Tree

Binary Tree	N-ary Tree
Maks 2 anak	Maks N anak
Left dan Right	Banyak Child
Lebih sederhana	Lebih fleksibel

Contoh:

Binary Tree:



N-ary Tree:



3. Istilah Penting

Root

Node paling atas.

A

Parent

Node yang memiliki anak.

A

Child

Node yang memiliki parent.

B C D E

Leaf

Node tanpa anak.

E F G H I

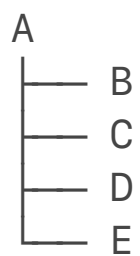
Sibling

Node dengan parent yang sama.

B C D

4. Representasi N-ary Tree

Misal:



Representasi:

```
struct Node{
```

```
    char data;

    struct Node* child[100];

    int childCount;

};
```

5. Struktur Node

```
struct Node{

    int id;

    struct Node* child[100];

    int childCount;

};
```

Setiap node menyimpan:

Data

Jumlah Child

Array Child

6. Membuat Node Baru

```
struct Node* createNode(int x){
```

```
struct Node* node =  
(struct Node*)malloc(sizeof(struct Node));  
  
node->id = x;  
  
node->childCount = 0;  
  
return node;  
  
}
```

7. Menambah Child

Misal:

A

ditambah:

B

Implementasi:

```
parent->child[parent->childCount]  
= child;
```

```
parent->childCount++;
```

Hasil:

A

|

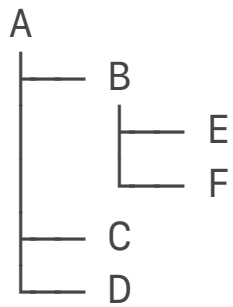
B

8. Traversal DFS

Depth First Search.

Kunjungi node lalu seluruh anaknya.

Contoh:



Hasil DFS:

A B E F C D

Implementasi:

```
void dfs(struct Node* root){  
  
    if(root == NULL)  
        return;  
  
    printf("%d ",root->id);
```

```

    for(int i=0;
        i<root->childCount;
        i++){

        dfs(root->child[i]);

    }

}

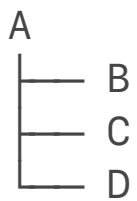
```

9. Traversal BFS

Breadth First Search.

Per level.

Contoh:



Hasil:

A B C D

Menggunakan Queue.

10. Menghitung Jumlah Node

```
int countNode(struct Node* root){

    if(root == NULL)
        return 0;

    int total = 1;

    for(int i=0;
        i<root->childCount;
        i++){

        total +=
            countNode(root->child[i]);

    }

    return total;

}
```

11. Menghitung Jumlah Leaf

```
if(root->childCount == 0)
maka:
Leaf
```

Implementasi rekursif.

12. Menghitung Tinggi Tree

```
int height(struct Node* root)
```

Cari tinggi terbesar dari seluruh child.

Rumus:

$1 + \text{tinggi maksimum child}$

13. Mencari Data

DFS:

```
search(root, x)
```

Periksa:

Root

Seluruh Child

secara rekursif.

14. Menampilkan Semua Level

Contoh:

Level 0 : A

Level 1 : B C D

Level 2 : E F G H

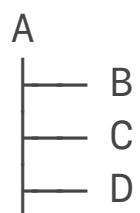
Menggunakan BFS.

15. Degree Node

Degree:

Jumlah child suatu node

Contoh:



Degree A:

3

16. Degree Tree

Degree tree:

Degree terbesar
dari seluruh node.

Contoh:

A memiliki 5 child

Maka:

Degree Tree = 5

17. Kompleksitas

Operasi	Kompleksitas
DFS	$O(n)$
BFS	$O(n)$
Search	$O(n)$
Count Node	$O(n)$
Height	$O(n)$

18. Kesalahan Umum

Child Count Tidak Bertambah

Salah:

```
parent->child[i] = x;
```

tanpa:

```
childCount++;
```

Akses Child Di Luar Batas

Salah:

```
child[100]
```

padahal ukuran:

```
0..99
```

Rekursi Tanpa Base Case

Salah:

```
dfs(root);
```

tanpa:

```
if(root == NULL)
```

Node Tidak Dialokasikan

Salah:

```
node->id = 10;
```

sebelum:

```
malloc()
```

19. Implementasi Lengkap

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node{

    int data;

    struct Node* child[100];

    int childCount;

};

struct Node* createNode(int x){

    struct Node* node =
    (struct Node*)malloc(sizeof(struct Node));

    node->data = x;

    node->childCount = 0;

    return node;

}

void addChild(struct Node* parent,
              struct Node* child){

    parent->child[parent->childCount]
    = child;

    parent->childCount++;

}

void dfs(struct Node* root){

    if(root == NULL)
        return;
```

```
printf("%d ",root->data);

for(int i=0;
    i<root->childCount;
    i++){

    dfs(root->child[i]);

}

}
```
